

CCTV Name Recognition & AI Security System

Comprehensive Technical Documentation, Architecture Guide, Directory Breakdown & Complete Route Catalog

Project: Smart Home CCTV Name Recognition	Stack: React.js Express FastAPI MongoDB
AI Engines: YOLOv8 Face & InsightFace ArcFace ONNX	Document Version: 1.0.0 (Production)

1. Executive Summary & System Architecture

The **CCTV Name Recognition & Security System** is a state-of-the-art, multi-tier security platform designed for real-time facial detection, high-accuracy face recognition, intruder detection, security alerts, and comprehensive audit reporting. The system operates locally over a microservice framework comprising three primary services and a central MongoDB database:

- Frontend Client (React.js - Port 3000):** Modern responsive web dashboard built with React, modular components, global dark-mode aesthetics, dynamic MJPEG stream renderer, real-time alert logs, face registration portal, camera management, and report exporter.
- Backend REST Server (Node.js/Express - Port 5000):** Core API server managing user authentication (JWT), Mongoose database models, camera configurations, multipart face image uploads via Multer, alert state transitions, and report generation (PDF, Excel, CSV).
- AI Computer Vision Microservice (Python FastAPI - Port 8000):** High-performance AI service executing multithreaded RTSP/Webcam frame capture, YOLOv8 face detection, InsightFace ArcFace ONNX (512-dimensional embedding extraction), cosine similarity search, intruder timer evaluation, automatic alert screenshot/video clip recording, and live MJPEG streaming.
- Database Layer (MongoDB - Port 27017):** Stores admin credentials, camera configurations, registered face vectors (512-dim arrays), historical detection logs, and security alert events.

Service Layer	Technology Stack	Port / Address	Primary Responsibilities
Frontend Client	React.js, Axios, React Router, CSS Glassmorphism	http://localhost:3000	User Dashboard, MJPEG video player, alerts grid, face registration UI, settings, exports.
Backend API	Node.js, Express, Mongoose, JWT, Multer, PDFKit, ExcelJS	http://localhost:5000	Authentication, CRUD endpoints, photo storage, alert status, report downloads.
AI Microservice	Python, FastAPI, OpenCV, YOLOv8, ONNX Runtime, PyMongo	http://localhost:8000	Camera RTSP/Webcam streams, YOLO face detection, ArcFace embedding, MJPEG streaming, intruder alerts.
Database	MongoDB, Mongoose ORM, PyMongo	mongodb://localhost:27017	Persistent storage for Users, Cameras, Faces (512-dim vectors), Alerts, and Detection Logs.

2. Detailed Workspace Directory & File Working Guide

The project repository is structured cleanly into three main execution folders (``ai-service/``, ``server/``, and ``client/``). Below is the explicit working and purpose of every single directory and component file across the codebase:

2.1 Root Directory (``/``)

- **`implementation_plan.md`**: Design architecture document outlining the complete multi-tier system plan, database schemas, verification tests, and operational workflows.
- **`CCTV_Name_Recognition_System_Documentation.pdf`**: Primary comprehensive technical documentation PDF (this file) detailing complete system working, directory structure, and route references.

2.2 Python AI Service (``ai-service/``)

The **`ai-service/`** directory contains the core computer vision engine built on FastAPI. It runs independently on **Port 8000** and directly interfaces with MongoDB and video hardware (Webcams / RTSP IP Cameras).

- **`main.py`**: Entry point of the Python AI microservice. Initializes FastAPI app with CORS middleware. Handles application startup and shutdown lifecycle events. Downloads model weights on boot, initializes ``EmbeddingManager``, ``FaceRecognizer``, and ``CameraRegistry``. Exposes REST API endpoints for embedding extraction, memory cache reloading, background camera stream sync, and live MJPEG video streaming.
- **`camera.py`**: Contains ``CameraStream`` and ``CameraRegistry`` classes. ``CameraStream`` runs a multithreaded continuous frame capture loop for RTSP/webcam sources. Maintains a 5-second circular frame buffer (``collections.deque``), executes face detection and ArcFace embedding comparison on every frame, draws green/red bounding boxes with names/scores, monitors intruder continuous detection timers, triggers automatic screenshot and MP4 video clip recording upon alert breach, and yields JPEG frames for live HTTP MJPEG streaming. ``CameraRegistry`` dynamically synchronizes running camera threads with active database camera configurations.
- **`face_detector.py`**: Provides the ``FaceDetector`` class. Implements face detection using YOLOv8-face (``yolov8n-face.pt``). If YOLO runtime or weights are unavailable, it automatically falls back seamlessly to an OpenCV DNN ResNet-SSD caffe face detector network to ensure uninterrupted system uptime.
- **`face_recognition.py`**: Provides the ``FaceRecognizer`` class. Loads the InsightFace ArcFace ONNX model (``w600k_r50.onnx``) using ``onnxruntime`` CPU provider. Resizes cropped face bounding boxes to 112x112 pixels, normalizes pixel intensity values, transposes to NCHW format, runs ONNX inference to generate a 512-dimensional floating-point embedding vector, and L2-normalizes the vector. Includes a static ``compare()`` method to compute cosine similarity between embedding vectors.
- **`embedding.py`**: Contains ``EmbeddingManager`` class. Directly connects to MongoDB using PyMongo. Loads registered face vector embeddings from ``faces`` collection into an in-memory cache for ultra-fast comparison. Provides helper functions to log detection events (``detectionlogs`` collection), record intruder security alerts (``alerts`` collection), and read system threshold settings.
- **`utils.py`**: Utility library providing file system management functions. Creates required upload and model directories (``models/``, ``../server/uploads/faces/``, ``../server/uploads/alerts/``). Automatically downloads pre-trained model weights (``yolov8n-face.pt`` and ``w600k_r50.onnx``) from remote storage if missing. Provides helper functions

``save_screenshot()`` and ``save_video()`` (which compiles frame buffers asynchronously into MP4 video files without blocking the main stream loop).

- **requirements.txt**: Defines Python dependencies: ``fastapi``, ``uvicorn``, ``opencv-python``, ``ultralytics``, ``onnxruntime``, ``pymongo``, ``requests``, and ``pydantic``.
- **models/**: Subdirectory serving as local storage for pre-trained AI weight files (``yolov8n-face.pt`` and ``w600k_r50.onnx``).

2.3 Backend REST API Server (``server/``)

The **server/** directory contains the Node.js & Express REST API server running on **Port 5000**. It manages authentication, business logic, file storage, and audit report generation.

- **server.js**: Main application entry point. Initializes Express, configures CORS, parses JSON and URL-encoded body payloads, ensures required ``uploads`` subdirectories exist, serves uploaded face and alert static media files via ``/uploads``, registers API route handlers, and sets up global error handling.
- **seed.js**: Database initial seeder script. Populates MongoDB with default admin credentials if no admin account exists.
- **package.json**: Node.js dependencies manifest detailing ``express``, ``mongoose``, ``jsonwebtoken``, ``bcryptjs``, ``multer``, ``cors``, ``dotenv``, ``pdfkit``, ``exceljs``, and ``csv-writer``.
- **config/db.js**: Establishes asynchronous connection to MongoDB database via Mongoose ORM using ``process.env.MONGODB_URI``.
- **middleware/auth.js**: Exports ``protect`` authentication middleware. Extracts Bearer JWT tokens from HTTP Authorization headers, verifies token signatures, fetches user context, and protects administrative endpoints from unauthorized access.
- **models/User.js**: Mongoose schema for system administrators. Stores ``name``, ``email``, ``password`` (hashed with ``bcrypt``), ``role``, and timestamps.
- **models/Camera.js**: Mongoose schema for CCTV cameras. Stores ``name``, ``url`` (webcam index '0' or RTSP URL), ``type`` (``webcam`` | ``rtsp``), ``location``, ``status`` (``online`` | ``offline``), and ``isActive`` boolean flag.
- **models/Face.js**: Mongoose schema for registered individuals. Stores ``name``, ``imagePaths`` array, 512-dimensional float ``embedding`` vector array, ``registeredBy`` User ID reference, and metadata.
- **models/Alert.js**: Mongoose schema for security intruder alerts. Stores ``camera`` ID reference, ``personName`` (``Unknown``), ``confidence`` score, ``screenshotPath``, ``videoPath``, ``status`` (``unread`` | ``read`` | ``resolved``), and timestamps.
- **models/DetectionLog.js**: Mongoose schema for continuous surveillance logging. Records every recognized individual or unknown face event per camera with timestamp and confidence.
- **controllers/authController.js**: Handles user login, registration, password updates, user profile retrieval, and system settings reads/updates.
- **controllers/cameraController.js**: Handles camera CRUD operations (listing cameras, adding new camera, updating configuration, toggling active state, deleting camera). Automatically notifies AI service to sync camera streams upon changes.
- **controllers/faceController.js**: Handles face registration. Receives uploaded photo files via Multer, saves them to disk, makes HTTP request to Python AI service (``http://localhost:8000/api/extract-embedding``) to extract 512-dim vector, stores face document with embedding in MongoDB, and triggers AI service cache reload (``/api/faces/reload``). Includes face deletion controller.

- **controllers/alertController.js**: Handles alert queries (fetching alerts with filtering and pagination), alert statistics calculation, marking alerts read/resolved, and deleting alert records.
- **controllers/reportsController.js**: Generates and streams downloadable audit log reports. Implements `downloadPDFReport` using `pdfkit`, `downloadExcelReport` using `exceljs`, and `downloadCSVReport` using `csv-writer`.
- **routes/authRoutes.js**: Express router for `/api/auth` endpoints (login, register, me, change-password, settings).
- **routes/cameraRoutes.js**: Express router for `/api/cameras` endpoints (camera listing, creation, modification, deletion).
- **routes/faceRoutes.js**: Express router for `/api/faces` endpoints with Multer image upload middleware (face registration, deletion).
- **routes/alertRoutes.js**: Express router for `/api/alerts` endpoints (fetching alerts, statistics, status updates, deletion).
- **routes/reportsRoutes.js**: Express router for `/api/reports` endpoints (PDF, Excel, CSV report downloads).
- **uploads/**: Static storage folder containing `/faces/` (registered photo assets) and `/alerts/` (intruder screenshots and MP4 video clips).

2.4 Frontend React Client (`client/`)

The **client/** directory houses the single-page React interface running on **Port 3000**. It delivers an intuitive dark-themed security command center.

- **public/index.html**: Main HTML shell document containing root DOM node `#root`, Google Font embeds (Inter / Outfit), and viewport meta tags.
- **src/index.js**: React application entry point. Mounts `App` root component into DOM inside `React.StrictMode` with `BrowserRouter`.
- **src/App.js**: Root layout component rendering the master `RoutesComponent`.
- **src/routes.js**: React Router v6 routing module defining view routes (`/login`, `/register`, `/dashboard`, `/cameras`, `/alerts`, `/reports`, `/settings`).
- **src/styles/global.css**: Comprehensive design system containing CSS variables, dark color palettes, glassmorphism cards, badges, buttons, modal overlays, layout grids, and responsive breakpoints.
- **src/services/api.js**: Configured Axios instance. Automatically injects standard base URL (`http://localhost:5000/api`) and attaches stored JWT Authorization headers to outgoing API requests.
- **src/components/Navbar.jsx**: Header navigation component displaying current system time, status indicators, active user profile, theme controls, and quick action buttons.
- **src/components/Sidebar.jsx**: Collapsible side navigation bar providing direct links to all operational modules with active route highlighting.
- **src/components/CameraCard.jsx**: Card component displaying live MJPEG video stream from Python AI service (`http://localhost:8000/api/cameras/{id}/stream`), camera metadata badges, full-screen toggle, and online status indicator.
- **src/components/StatCard.jsx**: Analytical overview widget displaying metrics (Total Cameras, Registered Faces, Active Alerts, Online Status) with icons and percentage changes.
- **src/components/AlertTable.jsx**: Tabular component displaying security alert logs with image thumbnail previews, severity badges, quick resolve buttons, and modal video player triggers.

- **src/components/ProtectedRoute.jsx**: Higher-order route wrapper verifying presence of valid JWT authentication token; redirects unauthenticated users to `/login`.
- **src/components/Loader.jsx**: Reusable visual loading spinner component for asynchronous data fetches.
- **src/pages/Login.jsx**: Admin authentication view with validation, error feedback, and localStorage JWT token persistence.
- **src/pages/Register.jsx**: User account registration view allowing creation of new administrator accounts.
- **src/pages/Dashboard.jsx**: Primary security overview dashboard featuring summary stat cards, live multi-camera stream grid, real-time alert feed, and quick system metrics.
- **src/pages/Cameras.jsx**: CCTV Camera management control page. Provides modals to add new webcams or RTSP streams, edit configurations, toggle camera activity, and delete cameras.
- **src/pages/FaceRegistration.jsx**: Face registration portal. Supports multi-photo file drag-and-drop or live webcam photo capture to register new recognized individuals in MongoDB.
- **src/pages/Alerts.jsx**: Dedicated security alerts audit portal with filter controls (all, unread, read), full screenshot preview, MP4 video player modal, and bulk resolve operations.
- **src/pages/Reports.jsx**: Audit reporting portal allowing custom date range filtering and one-click downloads of PDF, Excel, and CSV surveillance logs.
- **src/pages/Settings.jsx**: System configuration page to adjust facial recognition confidence thresholds (e.g. 0.55), intruder detection timers (e.g. 5.0s), and system preferences.
- **src/pages/NotFound.jsx**: Custom 404 Error fallback page for invalid route paths.

3. Comprehensive Route & Endpoint Catalog

Below is the complete reference catalog of all API endpoints and frontend routes across the three application tiers:

3.1 Python FastAPI AI Service Endpoints (Port 8000)

Method	Endpoint Path	Auth Required	Description & Data Payload
GET	/	None	Health check endpoint. Returns JSON status <code>{'status': 'online'}</code> .
POST	/api/extract-embedding	Internal	Payload: <code>{'image_path': str}</code> . Reads photo from disk, processes face crop through ArcFace ONNX, returns 512-dim embedding float array.
POST	/api/faces/reload	Internal	Triggers AI service to reload face vectors from MongoDB <code>'faces'</code> collection into local memory cache.
POST	/api/cameras/reload	Internal	Triggers background task to synchronize running camera stream threads (<code>'CameraRegistry'</code>) with active database camera configs.
GET	/api/cameras/{id}/stream	None	Streams live processed MJPEG video (<code>'multipart/x-mixed-replace'</code>) for specified camera with face bounding boxes, labels, scores, and intruder timer overlay.

3.2 Express Backend REST API Routes (Port 5000)

Module	Method & Route Path	Auth	Description & Functionality
Auth	POST /api/auth/register	None	Registers a new admin user in database with bcrypt password hashing.
Auth	POST /api/auth/login	None	Authenticates credentials, returns signed JWT token and user profile.
Auth	GET /api/auth/me	JWT	Fetches currently authenticated user details.
Auth	PUT /api/auth/change-password	JWT	Updates user password after verifying current password.
Auth	GET /api/auth/settings	JWT	Fetches system settings (recognition threshold, intruder timer).
Auth	POST /api/auth/settings	JWT	Updates system settings in database.
Cameras	GET /api/cameras	JWT	Returns list of all configured CCTV cameras.
Cameras	POST /api/cameras	JWT	Adds new webcam or RTSP camera config and syncs AI service streams.
Cameras	GET /api/cameras/:id	JWT	Retrieves camera details by ID.
Cameras	PUT /api/cameras/:id	JWT	Updates camera configuration details.
Cameras	DELETE /api/cameras/:id	JWT	Removes camera configuration from database and stops stream.
Faces	GET /api/faces	JWT	Fetches list of all registered faces.
Faces	POST /api/faces	JWT	Multer photo upload. Sends image to AI service to extract vector, saves Face in DB, reloads cache.
Faces	DELETE /api/faces/:id	JWT	Deletes registered face, removes photo files from disk, reloads AI cache.
Alerts	GET /api/alerts	JWT	Fetches intruder security alerts with pagination and filter criteria.
Alerts	GET /api/alerts/statistics	JWT	Calculates alert summary statistics for dashboard charts.
Alerts	PUT /api/alerts/:id/read	JWT	Marks an unread intruder alert as read/resolved.
Alerts	DELETE /api/alerts/:id	JWT	Deletes alert record and removes associated screenshot/video files.
Reports	GET /api/reports/pdf	JWT	Generates and streams a PDF audit report of detection/alert logs.
Reports	GET /api/reports/excel	JWT	Generates and streams an Excel (.xlsx) spreadsheet report.
Reports	GET /api/reports/csv	JWT	Generates and streams a CSV data export report.
Media	GET /uploads/*	None	Express static route serving stored face photos, alert screenshots, and MP4 video clips.

3.3 React Client Frontend Routes (Port 3000)

URL Path	Component Page	Protection	Function & User Workflow
/	Navigate	Public	Default root path. Automatically redirects user to `/login`.
/login	Login.jsx	Public	Admin login page. Authenticates user and saves JWT token to localStorage.
/register	Register.jsx	Public	User registration page for creating new administrator accounts.
/dashboard	Dashboard.jsx	Protected	Main operational dashboard displaying stat widgets, live camera stream grid, and alert ticker.
/cameras	Cameras.jsx	Protected	Camera manager view to register webcams or RTSP streams, edit settings, and toggle active streams.
/alerts	Alerts.jsx	Protected	Security intruder log view featuring thumbnail previews, modal MP4 video player, and resolve actions.
/reports	Reports.jsx	Protected	Surveillance report exporter view providing date filters and 1-click PDF, Excel, and CSV downloads.
/settings	Settings.jsx	Protected	System configuration view to adjust recognition thresholds and intruder detection timers.
*	NotFound.jsx	Public	Fallback 404 page for unmatched routes.

4. Step-by-Step Technical Workflows

Workflow A: Face Registration & Feature Vector Embedding Generation

- User uploads face photos or snaps photo via webcam on React `/face-registration` page.
- React POSTs multipart form data to Express `/api/faces` endpoint with JWT authentication header.
- Express Multer middleware validates JPEG/PNG image formats and saves image to `server/uploads/faces`.
- Express controller sends HTTP POST request to Python AI service (`http://localhost:8000/api/extract-embedding`) passing local image file path.
- Python AI service loads image using OpenCV, passes crop to InsightFace ArcFace ONNX model (`w600k_r50.onnx`), extracts 512-dim L2-normalized float array embedding, and returns vector JSON.
- Express stores Face record (Name, Photo Path, 512-dim Embedding Vector Array) in MongoDB `faces` collection.
- Express triggers POST to Python AI service (`/api/faces/reload`), updating in-memory face vector cache instantly without requiring service restart.

Workflow B: Real-Time Frame Capture, Detection & Cosine Similarity Recognition

- `CameraStream` thread continuously reads raw BGR frames from OpenCV `VideoCapture` (Webcam '0' or RTSP TCP URL).
- Raw frame is passed to `FaceDetector.detect()`, executing YOLOv8-face inference (or ResNet-SSD fallback) to locate bounding boxes `[x1, y1, x2, y2]`.

3. For each detected bounding box, the face crop is normalized to 112x112 and passed to ``FaceRecognizer.get_embedding()`` to compute a 512-dim vector.
4. Vector is compared against all registered face embeddings in memory using ``FaceRecognizer.compare()`` (dot product of L2-normalized vectors).
5. If maximum cosine similarity score exceeds ``recognition_threshold`` (e.g. 0.55), label is assigned person's registered Name and drawn in GREEN. Otherwise, label is set to 'Unknown' and drawn in RED.
6. Processed frame is encoded to JPEG format and streamed via HTTP MJPEG boundary generator to React `<img src='http://localhost:8000/api/cameras/{id}/stream' />` component.

Workflow C: Intruder Timer Tracking, Alert Media Recording & Security Notification

1. When an 'Unknown' face is continuously detected in frame, ``CameraStream`` initializes an ``unknown_detected_since`` timestamp.
2. If 'Unknown' presence duration reaches or exceeds ``intruder_timer`` (e.g. 5.0 seconds), intruder alert sequence activates.
3. System captures current raw frame and saves high-resolution screenshot to ``server/uploads/alerts/screenshot-{id}-{time}.jpg``.
4. Asynchronous worker thread (``save_video``) extracts last 5 seconds of frames from circular ``deque`` buffer and compiles MP4 video clip to ``server/uploads/alerts/video-{id}-{time}.mp4`` using OpenCV ``VideoWriter``.
5. ``EmbeddingManager.create_alert()`` creates an Alert record in MongoDB ``alerts`` collection with status ``unread``.
6. React client polls or fetches ``/api/alerts``, highlighting new alert badge and rendering screenshot preview & MP4 video modal in UI.

Workflow D: Surveillance Audit Logging & Report Generation

1. Recognized faces and intruder events are logged to MongoDB ``detectionlogs`` collection with camera reference, timestamp, and confidence score.
2. User navigates to React ``/reports`` page and selects desired date range and file format (PDF, Excel, CSV).
3. Express ``/api/reports/{format}`` endpoint queries Mongoose ``DetectionLog`` and ``Alert`` records matching filter.
4. Express compiles data stream dynamically using ``PDFKit`` (for formatted PDF), ``ExcelJS`` (for styled XLSX worksheets), or ``csv-writer`` (for CSV tables).
5. Report stream is delivered directly to user's browser with ``Content-Disposition: attachment`` header for instant downloading.

End of Documentation — CCTV Name Recognition & Security System